



EDUARDO ROMERA MARTÍNEZ

THUNDER INTELLIGENCE

War Thunder telemetry.
Continuous analytics. Results on the web.

PYTHON / KAFKA / FLINK / CONFLUENT CLOUD

educypher.gt.tc

Gameplay produces data

I wanted to capture what happens during gameplay and keep it in a system I could analyse and query.

CAPTURE

Vehicle readings and map positions.

PROCESS

Statistics updated as new readings arrive.

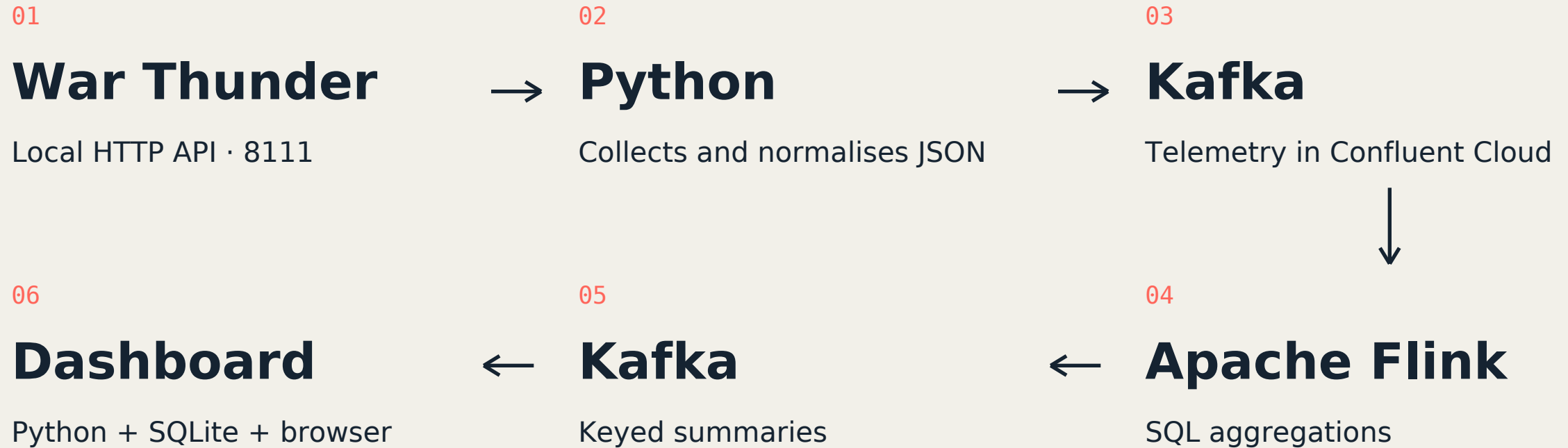
EXPLORE

Results organised by vehicle and session.



The path of a reading

Local capture, cloud processing and a browser dashboard.



The portfolio hosts the project documentation; the demonstrated dashboard runs locally.

Capture readings. Keep the context.

My collector reads War Thunder's local HTTP interface and publishes normalised observations to Kafka.

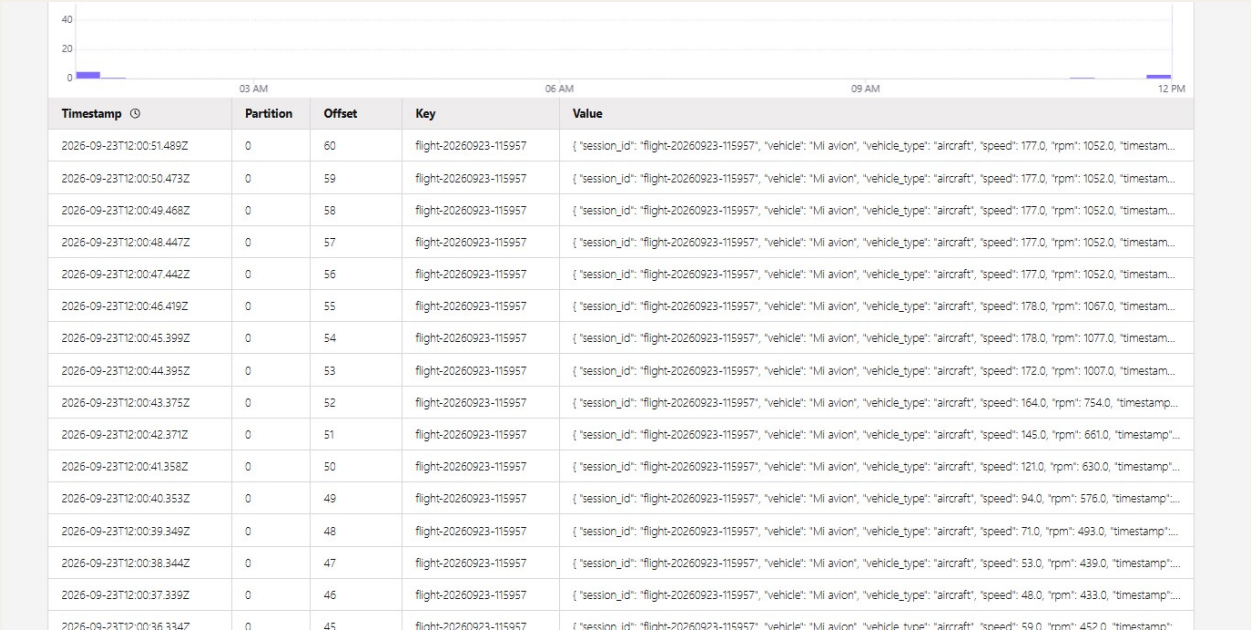
<code>/state</code>	Flight telemetry
<code>/indicators</code>	Vehicle indicators
<code>/map_obj.json</code>	Map position
<code>/hudmsg</code>	HUD events

Raw responses preserved in JSONL. Missing fields represented as null.



Kafka stores and distributes readings

I separated telemetry, events and results into different topics. Schema Registry defines the fields Flink can query.



Timestamp	Partition	Offset	Key	Value
2026-09-23T12:00:51.489Z	0	60	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 177.0, "rpm": 1052.0, "timestamp": ...
2026-09-23T12:00:50.473Z	0	59	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 177.0, "rpm": 1052.0, "timestamp": ...
2026-09-23T12:00:49.468Z	0	58	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 177.0, "rpm": 1052.0, "timestamp": ...
2026-09-23T12:00:48.447Z	0	57	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 177.0, "rpm": 1052.0, "timestamp": ...
2026-09-23T12:00:47.442Z	0	56	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 177.0, "rpm": 1052.0, "timestamp": ...
2026-09-23T12:00:46.419Z	0	55	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 178.0, "rpm": 1067.0, "timestamp": ...
2026-09-23T12:00:45.399Z	0	54	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 178.0, "rpm": 1077.0, "timestamp": ...
2026-09-23T12:00:44.395Z	0	53	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 172.0, "rpm": 1007.0, "timestamp": ...
2026-09-23T12:00:43.375Z	0	52	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 164.0, "rpm": 754.0, "timestamp": ...
2026-09-23T12:00:42.371Z	0	51	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 145.0, "rpm": 661.0, "timestamp": ...
2026-09-23T12:00:41.358Z	0	50	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 121.0, "rpm": 630.0, "timestamp": ...
2026-09-23T12:00:40.353Z	0	49	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 94.0, "rpm": 576.0, "timestamp": ...
2026-09-23T12:00:39.349Z	0	48	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 71.0, "rpm": 493.0, "timestamp": ...
2026-09-23T12:00:38.344Z	0	47	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 53.0, "rpm": 439.0, "timestamp": ...
2026-09-23T12:00:37.339Z	0	46	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 48.0, "rpm": 433.0, "timestamp": ...
2026-09-23T12:00:36.334Z	0	45	flight-20260923-115957	{ "session_id": "flight-20260923-115957", "vehicle": "Mi avion", "vehicle_type": "aircraft", "speed": 59.0, "rpm": 452.0, "timestamp": ...

JSON SCHEMA

session_id
vehicle
vehicle_type
speed · rpm
timestamp

Evidence: messages received by Kafka during project testing.

SQL that keeps processing data

I grouped readings by vehicle and session. Flink updates the summary as new messages arrive.

```
SELECT vehicle, session_id,  
       COUNT(*) AS total_lecturas,  
       AVG(speed) AS velocidad_media,  
       MAX(speed) AS velocidad_maxima  
FROM `warthunder-telemetry-v2`  
GROUP BY vehicle, session_id;
```

UPSERT

A composite key keeps the latest summary for each vehicle and session.

The average describes the collected samples; a session identifies one collector run.

Statistics in one view



PYTHON

Consumes the results topic.

SQLITE

Keeps the latest summary for each session.

Screenshot of the validated prototype. The local HTTP server runs at 127.0.0.1:8765.

Flight telemetry tests

F-20A / F-14D

30

samples · F-20A

1.02

maximum Mach · F-20A

27/30

positions · F-14D

Documented test sessions. The image illustrates aviation; it does not identify the measured vehicle.

The same pipeline, a ground vehicle

LEOPARD 1A5

I tested data capture during a test drive and followed the readings through to the dashboard.

30

telemetry samples

30

recorded positions

Speed source: `indicators.speed`.

Ground speed requires its own labels. The photograph is illustrative.



Results I could verify

Four collector runs recorded in the technical documentation.

Vehicle	Samples	Max. Mach	Positions	Type
F-20A	20	0.86	20 / 20	Air
F-20A	30	1.02	30 / 30	Air
F-14D	30	0.70	27 / 30	Air
Leopard 1A5	30	—	30 / 30	Ground

Each row is a collector session, not a complete battle. Positions are normalised map coordinates.

Decisions that made the system more reliable

Missing data

I preserved null values instead of inventing readings.

Updated results

I keyed summaries by vehicle + session_id.

Debugging by layer

I checked the API, Kafka, SQL and browser separately.



What works and what comes next

VALIDATED

Aircraft and Leopard 1A5 data capture.
Kafka messages and Flink aggregates.
Session results in the local dashboard.

NEXT STEPS

Detect battle boundaries reliably.
Expand ground metrics and vehicle coverage.
Complete imagery and public deployment.



Explore the full project

Project code, documentation and supporting material on my portfolio.



educypher.gt.tc

Scan the QR or open the website.
Find Thunder Intelligence and select
“Ver proyecto” (View project).

EDUARDO ROMERA MARTÍNEZ

